Séance 3

Les structures de contrôle

Objectifs de la séance

- Structurer le code en blocs grâce à l'indentation
- Écrire des conditions avec `if` / `elif` / `else` et l'opérateur ternaire
- Utiliser tous les opérateurs : comparaison, logiques, appartenance, identité
- Répéter avec `while` et `for` (`range`, `enumerate`, `zip`)
- Contrôler les boucles : `break`, `continue`, `pass`, `else`

⚡ Quiz flash : séance 2

1. Que vaut `"srv-web-01"[4:7]` ?
2. Que renvoie `type(input("Âge : "))` ?
3. Comment créer un ensemble vide ?
4. `a = [1, 2]`, `b = a`, `b.append(3)` : que vaut `a` ?
5. Écrire en une ligne l'échange des valeurs de `x` et `y`.

Plan de la séance

Durée	Bloc
10 min	Quiz flash
20 min	Théorie 1 : indentation, conditions, opérateurs
30 min	🧪 Lab partie 1
5 min	Pause
20 min	Théorie 2 : boucles `while` et `for`, branchements
30 min	🧪 Lab partie 2 + GestParc étape 2 : menu console
5 min	Récapitulatif

L'indentation fait partie de la syntaxe

```
disk_usage = 0.93
if disk_usage > 0.9:
    print("Disque presque plein")      # bloc du if
    print("Nettoyage des logs requis")  # même bloc
print("Contrôle terminé")              # hors du bloc
```

- Un bloc commence après `:` et est **indenté de 4 espaces** (PEP 8)
- Toutes les lignes d'un bloc ont la même indentation
- Les blocs s'imbriquent : 4 espaces de plus par niveau
- Ne jamais mélanger tabulations et espaces

`if` / `elif` / `else`

```
disk_usage = 0.78

if disk_usage >= 0.9:
    level = "CRITICAL"
elif disk_usage >= 0.7:
    level = "WARNING"
else:
    level = "OK"

print(f"Disque à {disk_usage:.0%} : {level}")
```

```
Disque à 78% : WARNING
```

Les conditions sont testées **dans l'ordre** : la première vraie l'emporte.

L'opérateur ternaire

```
is_online = False
status = "en ligne" if is_online else "hors ligne"

ram_gb = 6
advice = "OK" if ram_gb >= 8 else "Mise à niveau conseillée"
print(status, "-", advice)
```

```
hors ligne - Mise à niveau conseillée
```

`valeur_si_vrai if condition else valeur_si_faux`

💡 À réserver aux cas simples : sinon, un `if` classique est plus lisible.

Opérateurs de comparaison

Opérateur	Sens	Exemple	Résultat
`==` / `!=`	égal / différent	`"web" == "Web"`	`False`
`< <= > >=`	ordre	`"apple" < "banana"`	`True`
chaînage	encadrement	`0 <= port < 1024`	port système ?

```
port = 443
print(0 <= port < 1024, port == 443, "10" < "9")
```

```
True True True
```

⚠️ Les chaînes se comparent caractère par caractère : `"10" < "9"`.

Opérateurs logiques et valeurs « vraies »

Opérateur	Vrai si…
`a and b`	les deux sont vrais
`a or b`	au moins un est vrai
`not a`	`a` est faux

Valeurs considérées comme fausses : `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `set()`

```
tickets = []
if not tickets:
    print("Aucun ticket ouvert")
```

Évaluation **paresseuse** : `user and user["name"]` ne lit pas la clé si `user` est vide.

Appartenance et identité

```
ALLOWED_PORTS = {22, 80, 443}
port = 8080
print(port in ALLOWED_PORTS, port not in ALLOWED_PORTS)

laptop = {"asset_tag": "LT-001", "assigned_to": None}
print("brand" in laptop)              # cherche dans les CLÉS
if laptop["assigned_to"] is None:
    print("Équipement disponible")
```

```
False True
False
Équipement disponible
```

- `in` : liste, tuple, str (sous-chaîne), set (rapide), dict (clés)
- `is` / `is not` : **même objet** ; à utiliser surtout avec `None`

🧪 À vous : Lab partie 1 (30 min)

Notebook `S03_lab_enonce.ipynb`, partie 1

- Ex. 1 ⭐ : niveau d'alerte d'un disque
- Ex. 2 ⭐ : statut avec l'opérateur ternaire
- Ex. 3 ⭐⭐ : politique de mots de passe
- Ex. 4 ⭐⭐ : règles de pare-feu (appartenance, identité)
- Ex. 5 ⭐⭐⭐ : priorité d'un ticket (conditions imbriquées)

La boucle `while`

Répéter **tant qu'une condition est vraie**.

```
attempts = 0
delay = 1
while attempts < 4:
    attempts += 1
    print(f"Tentative {attempts} : nouvel essai dans {delay} s")
    delay *= 2
```

```
Tentative 1 : nouvel essai dans 1 s
Tentative 2 : nouvel essai dans 2 s
Tentative 3 : nouvel essai dans 4 s
Tentative 4 : nouvel essai dans 8 s
```

⚠️ La condition doit finir par devenir fausse : sinon, boucle infinie (Ctrl+C).

La boucle `for` et `range()`

Parcourir **chaque élément** d'une séquence.

```
servers = ["srv-web-01", "srv-db-01", "srv-app-01"]
for server in servers:
    print("Sauvegarde de", server)

for i in range(3):              # 0, 1, 2
    print(i, end=" ")
print()
for port in range(8000, 8010, 3):
    print(port, end=" ")
```

```
Sauvegarde de srv-web-01
Sauvegarde de srv-db-01
Sauvegarde de srv-app-01
0 1 2
8000 8003 8006 8009
```

`enumerate()` et `zip()`

```python
servers = ["srv-web-01", "srv-db-01"]
loads = [0.42, 0.91]

for index, server in enumerate(servers, start=1):
    print(index, server)

for server, load in zip(servers, loads):
    print(f"{server}: {load:.0%}")
```

```
1 srv-web-01
2 srv-db-01
srv-web-01: 42%
srv-db-01: 91%
```

Parcourir un dictionnaire

```
stock = {"laptop": 48, "server": 6, "printer": 9}

for category, count in stock.items():
    print(f"{category:<8} {count:>3}")

total = 0
for count in stock.values():
    total += count
print("Total :", total)
```

```
laptop    48
server     6
printer    9
Total : 63
```

`break`, `continue`, `pass`

```
log_lines = ["# en-tête", "INFO start", "ERROR disk", "FATAL crash", "INFO end"]
for line in log_lines:
    if line.startswith("#"):
        continue            # ligne suivante
    if line.startswith("FATAL"):
        print("Arrêt de l'analyse :", line)
        break               # sortie de la boucle
    if line.startswith("INFO"):
        pass                # rien à faire (pour l'instant)
    else:
        print("À traiter :", line)
```

```
À traiter : ERROR disk
Arrêt de l'analyse : FATAL crash
```

`else` sur une boucle

Le bloc `else` s'exécute si la boucle se termine **sans `break`**.

```
used_ips = {"10.0.0.10", "10.0.0.11", "10.0.0.13"}
for last_byte in range(10, 15):
    candidate = f"10.0.0.{last_byte}"
    if candidate not in used_ips:
        print("Première IP libre :", candidate)
        break
else:
    print("Aucune IP libre dans la plage")
```

```
Première IP libre : 10.0.0.12
```

Pièges fréquents

Piège	Exemple	Correction
`=` au lieu de `==`	`if level = "OK":`	`if level == "OK":`
Oubli des `:`	`else`	`else:`
Boucle infinie	compteur jamais incrémenté	vérifier la progression
Modifier la liste parcourue	`for x in l: l.remove(x)`	parcourir une copie `l[:]`
`range(len(l))`	`for i in range(len(l)): l[i]`	`for x in l` ou `enumerate`
`or` mal utilisé	`if level == "ERROR" or "FATAL":`	`if level in ("ERROR", "FATAL"):`

🧪 À vous : Lab partie 2 (30 min)

Notebook `S03_lab_enonce.ipynb`, parties 2 et 3

- Ex. 6 ⭐ : capacité totale d'un cluster (`for`)
- Ex. 7 ⭐⭐ : liste numérotée (`enumerate`)
- Ex. 8 ⭐⭐ : serveurs surchargés (`zip`)
- Ex. 9 ⭐⭐ : nouvelles tentatives avec attente croissante (`while`)
- Ex. 10 ⭐⭐⭐ : première IP libre (`break` / `else`)
- Synthèse : statistiques d'un journal d'événements
- **GestParc – étape 2** : menu console en boucle

Récapitulatif

- Blocs délimités par `:` et l'**indentation** (4 espaces)
- `if` / `elif` / `else`, ternaire `a if cond else b`
- Opérateurs : comparaison (chaînables), `and` / `or` / `not`, `in`, `is None`
- `while` : tant que ; `for` : pour chaque élément
- `range`, `enumerate`, `zip`, `dict.items()`
- `break`, `continue`, `pass`, `else` de boucle

Séance 4 (TP) : algorithmes classiques et automates à états